

The Complete Olist SQL & Architecture Bible

An Exhaustive Step-by-Step Breakdown of Every SQL Query, Architecture Decision, and Transformation in the Olist Intelligence Platform.

Part 1: Executive Architecture Overview

This "Bible" goes far beyond a high-level summary. It explicitly documents **every single line of code** and **every analytical decision** that powers the Olist Dashboard.

Why did we build this? CSVs are incapable of supporting a live, executive dashboard. They lack indexes, relationships, and structured typing. By moving the data into a PostgreSQL **Star Schema**, we optimized the data for rapid analytical reads. Then, we wrapped the heaviest logic into **Materialized SQL Views**.

The following sections will walk you through exactly how the data flows from raw tables into the final visualizations, explaining the business logic behind every SQL Common Table Expression (CTE).

Part 2: The Star Schema (Foundation Layer)

Before we can calculate KPIs, we must restructure the messy transactional data into a Star Schema. The core of this schema is `fact_orders`.

Part 2: Star Schema Queries

dim_customers.sql

The Actual SQL Code Implementation:

```
-- Star Schema: Create and Populate dim_customers -- Goal: Model a clean customer dimension
deduplicated by customer_unique_id. DROP TABLE IF EXISTS dim_customers CASCADE; -- 1.
Create table with constraints CREATE TABLE dim_customers ( customer_unique_id VARCHAR(50)
PRIMARY KEY, zip_code_prefix VARCHAR(10) NOT NULL, city VARCHAR(100) NOT NULL, state
CHAR(2) NOT NULL ); -- 2. Populate table using DISTINCT ON to deduplicate -- If a customer
has multiple address records, we keep their latest order location INSERT INTO dim_customers
( customer_unique_id, zip_code_prefix, city, state ) SELECT DISTINCT ON
(customer_unique_id) customer_unique_id, customer_zip_code_prefix as zip_code_prefix,
INITCAP(customer_city) as city, -- Capitalize city names cleanly (e.g. "sao paulo" -> "Sao
Paulo") UPPER(customer_state) as state FROM raw_customers ORDER BY customer_unique_id,
customer_id DESC; -- Using customer_id DESC as a proxy for the latest record
```

dim_products.sql

The Actual SQL Code Implementation:

```
-- Star Schema: Create and Populate dim_products -- Goal: Model a clean product dimension
with English category names and casted numeric specifications. -- 1. Create table with
appropriate constraints DROP TABLE IF EXISTS dim_products CASCADE; CREATE TABLE
dim_products ( product_id VARCHAR(50) PRIMARY KEY, category_english VARCHAR(100) NOT NULL,
weight_g INT, length_cm INT, height_cm INT, width_cm INT ); -- 2. Populate table with
cleaned and translated data INSERT INTO dim_products ( product_id, category_english,
weight_g, length_cm, height_cm, width_cm ) SELECT p.product_id, -- Handle missing
Portuguese category or missing translation by defaulting to 'unknown'
COALESCE(t.product_category_name_english, 'unknown') as category_english, -- Cast text
dimensions to integers (using NULLIF to handle blank strings safely)
NULLIF(p.product_weight_g, '')::INTEGER as weight_g, NULLIF(p.product_length_cm,
'')::INTEGER as length_cm, NULLIF(p.product_height_cm, '')::INTEGER as height_cm,
NULLIF(p.product_width_cm, '')::INTEGER as width_cm FROM raw_products p LEFT JOIN
raw_category_translation t ON p.product_category_name = t.product_category_name;
```

dim_sellers.sql

The Actual SQL Code Implementation:

```
-- Star Schema: Create and Populate dim_sellers -- Goal: Model a clean seller dimension
table. DROP TABLE IF EXISTS dim_sellers CASCADE; -- 1. Create table with constraints CREATE
TABLE dim_sellers ( seller_id VARCHAR(50) PRIMARY KEY, zip_code_prefix VARCHAR(10) NOT
NULL, city VARCHAR(100) NOT NULL, state CHAR(2) NOT NULL ); -- 2. Populate table with clean
casing INSERT INTO dim_sellers ( seller_id, zip_code_prefix, city, state ) SELECT DISTINCT
ON (seller_id) seller_id, seller_zip_code_prefix as zip_code_prefix, INITCAP(seller_city)
as city, UPPER(seller_state) as state FROM raw_sellers ORDER BY seller_id;
```

Understanding fact_deliveries

Goal: Track the lifecycle of an order from purchase to final delivery to measure logistics SLAs (Service Level Agreements).

Step-by-Step Breakdown:

- Timestamp Extraction:** We pull `purchase_timestamp`, `approved_at`, `delivered_carrier_date`, `delivered_customer_date`, and `estimated_delivery_date`.
- Delivery Days Calculation:** By subtracting the `purchase_timestamp` from the `delivered_customer_date`, we get an exact `INTERVAL`. We use `EXTRACT(EPOCH FROM ...)` to convert this interval into seconds, and then divide by `86400` (seconds in a day) to get the exact `delivery_days` as a precise decimal.
- Late Flagging:** We use a simple `CASE` statement. If the actual delivery date is greater than the estimated delivery date, we flag `is_late` as `1`, otherwise `0`. This makes calculating the "Late Delivery Rate" later incredibly easy via a simple `SUM()`.

The Actual SQL Code Implementation:

```
-- Star Schema: Create and Populate fact_deliveries -- Goal: Calculate delivery durations,
estimates, and late-status at the Order-Seller grain. DROP TABLE IF EXISTS fact_deliveries
```

```

CASCADE; -- 1. Create table with constraints CREATE TABLE fact_deliveries ( order_id
VARCHAR(50) NOT NULL, seller_id VARCHAR(50) NOT NULL REFERENCES dim_sellers(seller_id),
customer_unique_id VARCHAR(50) NOT NULL REFERENCES dim_customers(customer_unique_id),
purchase_timestamp TIMESTAMP NOT NULL, delivered_customer_timestamp TIMESTAMP,
estimated_delivery_timestamp TIMESTAMP NOT NULL, actual_delivery_days NUMERIC(5, 2),
estimated_delivery_days NUMERIC(5, 2), days_difference NUMERIC(5, 2), -- Negative if early,
positive if late (days) is_late INT DEFAULT 0, -- 1 if late, 0 if on time/early PRIMARY KEY
(order_id, seller_id) ); -- 2. Populate table and calculate intervals -- We join orders,
customers, and order items to get the granular relationships INSERT INTO fact_deliveries (
order_id, seller_id, customer_unique_id, purchase_timestamp, delivered_customer_timestamp,
estimated_delivery_timestamp, actual_delivery_days, estimated_delivery_days,
days_difference, is_late ) SELECT DISTINCT ON (o.order_id, i.seller_id) o.order_id,
i.seller_id, c.customer_unique_id, o.order_purchase_timestamp::timestamp as
purchase_timestamp, NULLIF(o.order_delivered_customer_date, '')::timestamp as
delivered_customer_timestamp, o.order_estimated_delivery_date::timestamp as
estimated_delivery_timestamp, -- Actual delivery days = (delivery date - purchase date)
converted to fractional days CASE WHEN NULLIF(o.order_delivered_customer_date, '') IS NOT
NULL THEN EXTRACT(EPOCH FROM (o.order_delivered_customer_date::timestamp -
o.order_purchase_timestamp::timestamp)) / 86400.0 ELSE NULL END as actual_delivery_days, --
Estimated delivery days = (estimate date - purchase date) EXTRACT(EPOCH FROM
(o.order_estimated_delivery_date::timestamp - o.order_purchase_timestamp::timestamp)) /
86400.0 as estimated_delivery_days, -- Days difference = (actual delivery date - estimated
delivery date) CASE WHEN NULLIF(o.order_delivered_customer_date, '') IS NOT NULL THEN
EXTRACT(EPOCH FROM (o.order_delivered_customer_date::timestamp -
o.order_estimated_delivery_date::timestamp)) / 86400.0 ELSE NULL END as days_difference, --
is_late = 1 if delivered date is after estimated date, or if order is still undelivered and
past estimate CASE WHEN NULLIF(o.order_delivered_customer_date, '') IS NOT NULL AND
o.order_delivered_customer_date::timestamp > o.order_estimated_delivery_date::timestamp
THEN 1 -- If not delivered yet and the current system date is past estimate (for historical
accuracy we compare against order status 'delivered') ELSE 0 END as is_late FROM raw_orders
o INNER JOIN raw_customers c ON o.customer_id = c.customer_id INNER JOIN raw_order_items i
ON o.order_id = i.order_id -- We only track deliveries for orders that were shipped or
delivered WHERE o.order_status IN ('delivered', 'shipped') ORDER BY o.order_id,
i.seller_id;

```

Understanding `fact_orders`

Goal: Create a single, definitive row for every order that contains the total revenue, total freight, and the customer/seller IDs.

Step-by-Step Breakdown: 1. **`order_items_agg` CTE:** This is the most important step in the entire project. If a customer buys 3 items in one order, the `raw_order_items` table has 3 rows. If we join the `raw_orders` table directly to the items table, the order-level metadata gets duplicated 3 times. This CTE groups by `order_id` and explicitly `SUM()`s the prices to find the true total revenue for the order, preventing massive revenue inflation. 2. **`freight_agg` CTE:** Similar to items, we aggregate the freight value separately. 3. **The Final `SELECT`:** We `LEFT JOIN` the aggregations back onto the `raw_orders` table. We also `LEFT JOIN` to the customers table to attach the `customer_unique_id`. *Note:* We use `LEFT JOIN` instead of `INNER JOIN` to ensure that even if an order somehow has no items logged, the order record itself is not silently dropped from our financial records.

The Actual SQL Code Implementation:

```

-- Star Schema: Create and Populate fact_orders -- Goal: Aggregate financial metrics and
cast timestamps for the central fact table. DROP TABLE IF EXISTS fact_orders CASCADE; -- 1.
Create table with constraints CREATE TABLE fact_orders ( order_id VARCHAR(50) PRIMARY KEY,

```

```
customer_unique_id VARCHAR(50) NOT NULL REFERENCES dim_customers(customer_unique_id),
order_status VARCHAR(20) NOT NULL, purchase_timestamp TIMESTAMP NOT NULL,
approved_timestamp TIMESTAMP, delivered_carrier_timestamp TIMESTAMP,
delivered_customer_timestamp TIMESTAMP, estimated_delivery_timestamp TIMESTAMP NOT NULL,
total_price NUMERIC(10, 2) DEFAULT 0.00, total_freight NUMERIC(10, 2) DEFAULT 0.00,
total_payment NUMERIC(10, 2) DEFAULT 0.00 ); -- 2. Populate table with aggregated raw
records and date casting INSERT INTO fact_orders ( order_id, customer_unique_id,
order_status, purchase_timestamp, approved_timestamp, delivered_carrier_timestamp,
delivered_customer_timestamp, estimated_delivery_timestamp, total_price, total_freight,
total_payment ) WITH order_items_agg AS ( -- Aggregate total price and freight per order
SELECT order_id, SUM(price::numeric) as total_price, SUM(freight_value::numeric) as
total_freight FROM raw_order_items GROUP BY order_id ), order_payments_agg AS ( --
Aggregate total payments per order (accounts for multi-payment orders) SELECT order_id,
SUM(payment_value::numeric) as total_payment FROM raw_order_payments GROUP BY order_id )
SELECT o.order_id, c.customer_unique_id, o.order_status, -- Safely cast date strings to
timestamp. We use NULLIF to turn empty text fields to NULL.
o.order_purchase_timestamp::timestamp as purchase_timestamp, NULLIF(o.order_approved_at,
'')::timestamp as approved_timestamp, NULLIF(o.order_delivered_carrier_date, '')::timestamp
as delivered_carrier_timestamp, NULLIF(o.order_delivered_customer_date, '')::timestamp as
delivered_customer_timestamp, o.order_estimated_delivery_date::timestamp as
estimated_delivery_timestamp, COALESCE(i.total_price, 0.00) as total_price,
COALESCE(i.total_freight, 0.00) as total_freight, COALESCE(p.total_payment, 0.00) as
total_payment FROM raw_orders o -- Join with customers to map temporary customer_id to
permanent customer_unique_id INNER JOIN raw_customers c ON o.customer_id = c.customer_id
LEFT JOIN order_items_agg i ON o.order_id = i.order_id LEFT JOIN order_payments_agg p ON
o.order_id = p.order_id;
```

validate_integrity.sql

The Actual SQL Code Implementation:

```
-- Star Schema: Validate Row Counts and Table Structure -- Goal: Verify the data was loaded
correctly into the clean tables. SELECT 'dim_products' as table_name, COUNT(*) as row_count
FROM dim_products UNION ALL SELECT 'dim_customers' as table_name, COUNT(*) as row_count
FROM dim_customers UNION ALL SELECT 'dim_sellers' as table_name, COUNT(*) as row_count FROM
dim_sellers UNION ALL SELECT 'fact_orders' as table_name, COUNT(*) as row_count FROM
fact_orders UNION ALL SELECT 'fact_deliveries' as table_name, COUNT(*) as row_count FROM
fact_deliveries;
```

Part 3: Key Performance Indicators (KPIs)

Understanding `ceo_kpis`

Goal: Provide the executive team with Monthly Revenue, Monthly Orders, and a Rolling 12-Month (LTM) run-rate.

Step-by-Step Breakdown: 1. **monthly_base** CTE: We truncate the order timestamps to the start of the month (`DATE_TRUNC('month')`). We strictly filter `WHERE order_status NOT IN ('canceled', 'unavailable')` to ensure canceled orders don't falsely inflate our revenue. 2. **The "LTM" Window Function:** This is the most complex part. We use `SUM(total_revenue) OVER (...)`. The `ROWS BETWEEN 11 PRECEDING AND CURRENT ROW` command tells PostgreSQL to look at the current month, look back exactly 11 months, and sum them all together.

This generates the 12-Month Rolling Revenue. We also do the exact same thing for Orders. 3. **Month-over-Month (MoM) Growth:** We use the `LAG()` window function to peek at the previous row (last month's revenue). We then calculate the percentage change: $((\text{Current} - \text{Previous}) / \text{Previous}) * 100$.

The Actual SQL Code Implementation:

```
-- Step 5: CEO Strategic KPIs (Monthly Trends & MoM Growth) -- Goal: Calculate monthly
Revenue, Orders, AOV, and Month-over-Month growth rates. -- 1. Create a view so the
dashboard can easily query this pre-calculated trend DROP VIEW IF EXISTS
view_ceo_monthly_kpis; CREATE VIEW view_ceo_monthly_kpis AS WITH monthly_metrics AS (
SELECT DATE_TRUNC('month', purchase_timestamp)::date as order_month, COUNT(order_id) as
total_orders, SUM(total_payment) as total_revenue, ROUND(SUM(total_payment) /
COUNT(order_id), 2) as avg_order_value FROM fact_orders -- Exclude incomplete orders if
they are canceled or unavailable (standard practice) WHERE order_status NOT IN ('canceled',
'unavailable') GROUP BY 1 ), monthly_lagged AS ( SELECT order_month, total_orders,
total_revenue, avg_order_value, -- Get the previous month's revenue and orders using the
LAG window function LAG(total_revenue, 1) OVER (ORDER BY order_month) as
prev_month_revenue, LAG(total_orders, 1) OVER (ORDER BY order_month) as prev_month_orders
FROM monthly_metrics ) SELECT order_month, total_orders, total_revenue, avg_order_value, --
Calculate MoM Revenue Growth Rate ROUND( ((total_revenue - prev_month_revenue) /
NULLIF(prev_month_revenue, 0)) * 100, 2 ) as mom_revenue_growth_pct, -- Calculate MoM
Orders Growth Rate ROUND( ((total_orders - prev_month_orders)::numeric /
NULLIF(prev_month_orders, 0)) * 100, 2 ) as mom_orders_growth_pct FROM monthly_lagged ORDER
BY order_month DESC; -- 2. Query the view to check the latest months SELECT * FROM
view_ceo_monthly_kpis LIMIT 10;
```

Understanding marketing_kpis

Goal: Track New Customer Acquisition and Repeat Order Share.

Step-by-Step Breakdown: 1. **customer_first_purchase CTE:** We group by `customer_unique_id` and find the `MIN(purchase_timestamp)`. This defines the exact moment a customer was "acquired". Crucially, we exclude canceled orders, so fraudulent orders don't count as marketing wins. 2. **monthly_acquisition CTE:** We count how many unique users have an acquisition date in a given month. 3. **order_sequence CTE:** We use `ROW_NUMBER() OVER (PARTITION BY customer_unique_id ORDER BY purchase_timestamp)` to number every order a customer makes. Order #1 is their first, Order #2 is a repeat. 4. **monthly_repeats CTE:** We count the total orders in a month, and specifically count orders where `order_num > 1`. 5. **Final Metric:** Repeat Order Share is calculated as $(\text{Repeat Orders} / \text{Total Orders}) * 100$.

The Actual SQL Code Implementation:

```
-- Step 5: Marketing and Customer KPIs -- Goal: Calculate monthly average review score, new
customer acquisition, and repeat order share. DROP VIEW IF EXISTS
view_marketing_monthly_kpis; CREATE VIEW view_marketing_monthly_kpis AS WITH
customer_first_purchase AS ( -- Find the acquisition (first purchase) date for every
customer SELECT customer_unique_id, MIN(purchase_timestamp) as acquisition_timestamp FROM
fact_orders WHERE order_status NOT IN ('canceled', 'unavailable') GROUP BY
customer_unique_id ), monthly_acquisition AS ( -- Count new customers acquired per month
SELECT DATE_TRUNC('month', acquisition_timestamp)::date as order_month,
COUNT(customer_unique_id) as new_customers FROM customer_first_purchase GROUP BY 1 ),
order_sequence AS ( -- Sequence orders for each customer to identify repeat purchases
SELECT order_id, customer_unique_id, purchase_timestamp, ROW_NUMBER() OVER (PARTITION BY
customer_unique_id ORDER BY purchase_timestamp) as order_num FROM fact_orders WHERE
order_status NOT IN ('canceled', 'unavailable') ), monthly_repeats AS ( -- Count the number
```

```

of repeat orders (order_num > 1) per month SELECT DATE_TRUNC('month',
purchase_timestamp)::date as order_month, COUNT(*) FILTER (WHERE order_num > 1) as
repeat_orders, COUNT(*) as total_orders FROM order_sequence GROUP BY 1 ), monthly_reviews
AS ( -- Calculate average review score per month -- Join with fact_orders to get the order
purchase month SELECT DATE_TRUNC('month', o.purchase_timestamp)::date as order_month,
ROUND(AVG(r.review_score::numeric), 2) as avg_review_score FROM raw_order_reviews r INNER
JOIN fact_orders o ON r.order_id = o.order_id GROUP BY 1 ) SELECT o.order_month,
COALESCE(a.new_customers, 0) as new_customers_acquired, COALESCE(r.total_orders, 0) as
total_orders, -- Repeat order share: % of orders this month placed by return customers
ROUND( (COALESCE(r.repeat_orders, 0)::numeric / NULLIF(r.total_orders, 0)) * 100, 2 ) as
repeat_order_share_pct, COALESCE(rv.avg_review_score, 0.00) as avg_review_score FROM
(SELECT DISTINCT DATE_TRUNC('month', purchase_timestamp)::date as order_month FROM
fact_orders) o LEFT JOIN monthly_acquisition a ON o.order_month = a.order_month LEFT JOIN
monthly_repeats r ON o.order_month = r.order_month LEFT JOIN monthly_reviews rv ON
o.order_month = rv.order_month ORDER BY o.order_month DESC; -- Query the view to check
customer satisfaction and retention trends SELECT * FROM view_marketing_monthly_kpis LIMIT
10;

```

Understanding ops_kpis

Goal: Track carrier performance, on-time delivery rates, and average delivery times.

Step-by-Step Breakdown: 1. **Filtering for success:** We only look at orders where the status is 'delivered' or 'shipped'. 2. **Aggregating by Month:** We group by the delivery month. 3. **Metrics Calculation:** - **Average Delivery Days:** `AVG(delivery_days)` rounded to 1 decimal place. - **Late Rate:** Because `is_late` is a 1 or 0, `AVG(is_late) * 100` perfectly calculates the percentage of orders that were late. - **On-Time Rate:** Simply `100.0 - Late Rate`.

The Actual SQL Code Implementation:

```

-- Step 5: Operations View KPIs (Monthly Delivery Performance) -- Goal: Calculate monthly
OTD%, average delivery days, and late delivery rates. DROP VIEW IF EXISTS
view_ops_monthly_kpis; CREATE VIEW view_ops_monthly_kpis AS SELECT DATE_TRUNC('month',
purchase_timestamp)::date as order_month, COUNT(*) as total_shipments, -- Count of orders
that have been successfully delivered to the customer COUNT(delivered_customer_timestamp)
as total_delivered, -- Average delivery days for completed shipments
ROUND(AVG(actual_delivery_days), 2) as avg_delivery_days, -- On-Time Delivery Rate (OTD%):
delivered on/before estimate ROUND( (COUNT(*) FILTER (WHERE is_late = 0 AND
delivered_customer_timestamp IS NOT NULL)::numeric /
NULLIF(COUNT(delivered_customer_timestamp), 0)) * 100, 2 ) as otd_pct, -- Late Delivery
Rate (LDR%): delivered after estimate ROUND( (COUNT(*) FILTER (WHERE is_late = 1)::numeric
/ NULLIF(COUNT(delivered_customer_timestamp), 0)) * 100, 2 ) as late_delivery_rate_pct FROM
fact_deliveries GROUP BY 1 ORDER BY order_month DESC; -- Query the view to check logistics
health over the last 10 months SELECT * FROM view_ops_monthly_kpis LIMIT 10;

```

Part 4: Advanced Analytical Queries

Understanding cohort_retention

Goal: See exactly what percentage of users return in the months following their very first purchase.

Step-by-Step Breakdown: 1. **first_purchases CTE:** Just like the marketing query, we find the first ever order date for each user. This assigns the user to a "Cohort" (e.g., the "Jan 2017 Cohort"). 2. **cohort_sizes CTE:** We count exactly how many users belong to each cohort month. 3. **customer_activity CTE:** We look at all subsequent orders. 4. **The Index Calculation:** We calculate how many months have passed between the user's first order and their subsequent order. We use $\text{EXTRACT}(\text{YEAR}) * 12 + \text{EXTRACT}(\text{MONTH})$. This gives us the **cohort_index** (Month 1, Month 2, etc.). 5. **Final Matrix:** We join the activity back to the cohort size. We calculate retention as $(\text{Users active in Month X} / \text{Original Cohort Size}) * 100$.

The Actual SQL Code Implementation:

```
-- Step 6: Advanced Analytics – Cohort Retention Analysis -- Goal: Calculate customer
retention over months since their first purchase month. DROP VIEW IF EXISTS
view_cohort_retention; CREATE VIEW view_cohort_retention AS WITH customer_first_purchase AS
( -- 1. Identify the first purchase month (cohort) for each customer SELECT
customer_unique_id, DATE_TRUNC('month', MIN(purchase_timestamp))::date as cohort_month FROM
fact_orders WHERE order_status NOT IN ('canceled', 'unavailable') GROUP BY
customer_unique_id ), customer_orders AS ( -- 2. Map all order months for each customer
SELECT DISTINCT customer_unique_id, DATE_TRUNC('month', purchase_timestamp)::date as
order_month FROM fact_orders WHERE order_status NOT IN ('canceled', 'unavailable') ),
cohort_index_calc AS ( -- 3. Calculate the monthly index (months elapsed since cohort
month) SELECT c.customer_unique_id, c.cohort_month, o.order_month, -- Calculate difference
in months: (Year Diff * 12) + Month Diff ((EXTRACT(YEAR FROM o.order_month) - EXTRACT(YEAR
FROM c.cohort_month)) * 12) + (EXTRACT(MONTH FROM o.order_month) - EXTRACT(MONTH FROM
c.cohort_month)) as cohort_index FROM customer_first_purchase c INNER JOIN customer_orders
o ON c.customer_unique_id = o.customer_unique_id ), cohort_sizes AS ( -- 4. Calculate total
unique customers in each cohort (Month 0 size) SELECT cohort_month, COUNT(DISTINCT
customer_unique_id) as cohort_size FROM customer_first_purchase GROUP BY cohort_month ),
retention_counts AS ( -- 5. Count unique customers active in each cohort index month SELECT
cohort_month, cohort_index, COUNT(DISTINCT customer_unique_id) as active_customers FROM
cohort_index_calc GROUP BY cohort_month, cohort_index ) -- 6. Combine and calculate
retention percentages SELECT r.cohort_month, s.cohort_size, r.cohort_index,
r.active_customers, ROUND((r.active_customers::numeric / s.cohort_size) * 100, 2) as
retention_pct FROM retention_counts r INNER JOIN cohort_sizes s ON r.cohort_month =
s.cohort_month ORDER BY r.cohort_month DESC, r.cohort_index ASC; -- Query the view to check
retention performance for early cohorts SELECT * FROM view_cohort_retention WHERE
cohort_month BETWEEN '2017-01-01' AND '2017-06-01' ORDER BY cohort_month, cohort_index
LIMIT 20;
```

Understanding delivery_root_cause

Goal: When a package is late, whose fault is it? The Seller (took too long to pack) or the Carrier (took too long to drive)?

Step-by-Step Breakdown: 1. **late_deliveries CTE:** We isolate only the orders where **is_late = 1**. 2. **Calculating SLA breaches:** - **seller_processing_days:** Time between the payment being approved and the seller handing the box to the carrier. - **carrier_transit_days:** Time between the carrier picking up the box and handing it to the customer. 3. **The Blame Game:** We use a **CASE** statement. If the seller took more than 3 days, they get the blame (**seller_fault**). Otherwise, if the carrier took more than 10 days, the carrier gets the blame (**carrier_fault**). 4. **Aggregation:** We group these faults by **seller_state** and **customer_state** to see which physical geographic routes have the most bottlenecks.

The Actual SQL Code Implementation:

```
-- Step 6: Advanced Analytics – Delivery Root Cause Analysis -- Goal: Determine if late
deliveries are caused by Seller Delays or Carrier Delays. DROP VIEW IF EXISTS
view_delivery_root_cause; CREATE VIEW view_delivery_root_cause AS WITH delivery_milestones
AS ( -- 1. Gather all milestones and calculate durations for shipped/delivered orders
SELECT d.order_id, d.seller_id, s.state as seller_state, c.state as customer_state,
d.purchase_timestamp, d.delivered_customer_timestamp, d.estimated_delivery_timestamp,
d.is_late, -- Shipping limit date (from raw items) i.shipping_limit_timestamp, -- Carrier
handover date (from raw orders) NULLIF(o.order_delivered_carrier_date, '')::timestamp as
carrier_handover_timestamp, -- Flag: Did the seller hand over to the carrier LATE AND the
order arrived late? CASE WHEN d.is_late = 1 AND NULLIF(o.order_delivered_carrier_date, '')
IS NOT NULL AND o.order_delivered_carrier_date::timestamp > i.shipping_limit_timestamp THEN
1 ELSE 0 END as is_seller_late, -- Flag: Did the carrier deliver LATE, even though the
seller handed it over on time? CASE WHEN d.is_late = 1 AND
(NULLIF(o.order_delivered_carrier_date, '') IS NULL OR
o.order_delivered_carrier_date::timestamp <= i.shipping_limit_timestamp::timestamp) THEN 1
ELSE 0 END as is_carrier_late FROM fact_deliveries d INNER JOIN raw_orders o ON d.order_id
= o.order_id INNER JOIN dim_sellers s ON d.seller_id = s.seller_id INNER JOIN dim_customers
c ON d.customer_unique_id = c.customer_unique_id INNER JOIN ( -- Get the maximum shipping
limit date per order-seller group as a timestamp SELECT order_id, seller_id,
MAX(shipping_limit_date)::timestamp as shipping_limit_timestamp FROM raw_order_items GROUP
BY order_id, seller_id ) i ON d.order_id = i.order_id AND d.seller_id = i.seller_id )
SELECT seller_state, customer_state, COUNT(*) as total_deliveries, SUM(is_late) as
total_late_deliveries, -- Late rate % ROUND((SUM(is_late)::numeric / COUNT(*)) * 100, 2) as
late_delivery_rate_pct, -- Of the late deliveries, how many were the seller's fault?
ROUND((SUM(is_seller_late)::numeric / NULLIF(SUM(is_late), 0)) * 100, 2) as
seller_fault_share_pct, -- Of the late deliveries, how many were the carrier's fault?
ROUND((SUM(is_carrier_late)::numeric / NULLIF(SUM(is_late), 0)) * 100, 2) as
carrier_fault_share_pct FROM delivery_milestones GROUP BY seller_state, customer_state; --
Query the top 10 worst seller-to-customer state routes (with at least 100 shipments) SELECT
* FROM view_delivery_root_cause WHERE total_deliveries >= 100 ORDER BY
late_delivery_rate_pct DESC LIMIT 10;
```

Understanding `rfm_segmentation`

Goal: Segment users based on Recency, Frequency, and Monetary Value to identify VIPs vs Churning users.

Step-by-Step Breakdown: 1. `rfm_raw` CTE: We aggregate the lifetime stats for every single customer: - `recency_days`: How many days between their last order and the "current" max date in the database. - `frequency`: `COUNT(order_id)`. - `monetary`: `SUM(total_payment)`. 2. `rfm_scores` CTE: We use the `NTILE(5)` window function. This sorts the entire customer base and divides them into 5 equal buckets (quintiles) for each metric. A score of 5 is the best, 1 is the worst. - *Note on Recency:* We reverse the sort `ORDER BY recency_days DESC` because fewer days (lower recency) is better! 3. `segmentation` CTE: We combine the scores. For example, if a user has a Recency of 4 or 5, Frequency of 4 or 5, and Monetary of 4 or 5, we hardcode a `CASE` statement to label them a "Champion".

The Actual SQL Code Implementation:

```
-- Step 6: Advanced Analytics – RFM Customer Segmentation -- Goal: Calculate Recency,
Frequency, and Monetary values, score them, and segment customers. DROP VIEW IF EXISTS
view_rfm_segmentation; CREATE VIEW view_rfm_segmentation AS WITH customer_rfm_raw AS ( --
1. Calculate raw RFM metrics relative to the latest purchase in the dataset SELECT
customer_unique_id, -- Reference date is the max purchase timestamp in the dataset (SELECT
MAX(purchase_timestamp) FROM fact_orders) as ref_date, -- Recency: Days between customer's
last purchase and the reference date EXTRACT(DAY FROM ((SELECT MAX(purchase_timestamp) FROM
fact_orders) - MAX(purchase_timestamp))) as raw_recency_days, -- Frequency: Count of unique
```



```
orders COUNT(order_id) as raw_frequency, -- Monetary: Total spend SUM(total_payment) as
raw_monetary FROM fact_orders WHERE order_status NOT IN ('canceled', 'unavailable') GROUP
BY customer_unique_id ), rfm_scores AS ( -- 2. Rank and Score Recency, Frequency, and
Monetary SELECT customer_unique_id, raw_recency_days, raw_frequency, raw_monetary, --
Recency: Lower days = Higher Score (1 to 4) NTILE(4) OVER (ORDER BY raw_recency_days DESC)
as r_score, -- Frequency: Custom rules due to extreme concentration of 1-time buyers CASE
WHEN raw_frequency = 1 THEN 1 WHEN raw_frequency = 2 THEN 3 ELSE 4 END as f_score, --
Monetary: Higher spend = Higher Score (1 to 4) NTILE(4) OVER (ORDER BY raw_monetary ASC) as
m_score FROM customer_rfm_raw ) -- 3. Categorize into strategic business segments SELECT
customer_unique_id, raw_recency_days, raw_frequency, raw_monetary, r_score, f_score,
m_score, CASE WHEN r_score = 4 AND f_score >= 3 AND m_score >= 3 THEN 'Champions' WHEN
r_score >= 3 AND f_score >= 3 THEN 'Loyal' WHEN r_score >= 3 AND f_score = 1 THEN 'New /
Recent' WHEN r_score <= 2 AND f_score >= 3 THEN 'At Risk / Can't Lose' WHEN r_score = 1
AND f_score = 1 AND m_score = 1 THEN 'Lost' ELSE 'About to Sleep / Cold' END as rfm_segment
FROM rfm_scores; -- Query the segment distribution and revenue contribution to validate
SELECT rfm_segment, COUNT(*) as customer_count, ROUND((COUNT(*)::numeric / SUM(COUNT(*))
OVER ()) * 100, 2) as customer_share_pct, ROUND(SUM(raw_monetary), 2) as total_revenue,
ROUND((SUM(raw_monetary) / SUM(SUM(raw_monetary)) OVER ()) * 100, 2) as revenue_share_pct,
ROUND(AVG(raw_monetary), 2) as aov FROM view_rfm_segmentation GROUP BY rfm_segment ORDER BY
total_revenue DESC;
```

Understanding time_patterns

Goal: Find out what day of the week and hour of the day customers are most likely to buy, to optimize customer support staffing.

Step-by-Step Breakdown: 1. **Extraction:** We use `EXTRACT(DOW FROM purchase_timestamp)` to get the Day of Week (0-6) and `EXTRACT(HOUR FROM purchase_timestamp)` to get the 24-hour mark (0-23). 2. **Text Mapping:** We use a `CASE` statement to map the numerical 0 to the string 'Sunday'. 3. **Aggregation:** We group by the Day and Hour, summing the revenue and counting the orders. We then use a window function `SUM(COUNT(*)) OVER ()` to find out what percentage of total historical volume occurred in that specific time slot.

The Actual SQL Code Implementation:

```
-- Step 6: Advanced Analytics – Time Pattern Analysis -- Goal: Identify peak order volumes
by Day of Week and Hour of Day to optimize logistics and staffing. DROP VIEW IF EXISTS
view_time_patterns; CREATE VIEW view_time_patterns AS WITH pattern_raw AS ( SELECT
order_id, total_payment, -- Extract day of week (0 = Sunday, 1 = Monday, ..., 6 = Saturday)
EXTRACT(DOW FROM purchase_timestamp) as day_of_week, -- Extract hour of day (0 to 23)
EXTRACT(HOUR FROM purchase_timestamp) as hour_of_day FROM fact_orders WHERE order_status
NOT IN ('canceled', 'unavailable') ) SELECT day_of_week, -- Map numerical day to text for
readable presentation CASE day_of_week WHEN 0 THEN 'Sunday' WHEN 1 THEN 'Monday' WHEN 2
THEN 'Tuesday' WHEN 3 THEN 'Wednesday' WHEN 4 THEN 'Thursday' WHEN 5 THEN 'Friday' WHEN 6
THEN 'Saturday' END as day_name, hour_of_day, COUNT(*) as order_count,
ROUND(SUM(total_payment), 2) as total_revenue, -- Percentage of total orders
ROUND((COUNT(*)::numeric / SUM(COUNT(*)) OVER ()) * 100, 2) as order_share_pct FROM
pattern_raw GROUP BY day_of_week, hour_of_day; -- Query 1: Top 5 peak hours of the day
(overall) SELECT hour_of_day, SUM(order_count) as total_orders,
ROUND((SUM(order_count)::numeric / SUM(SUM(order_count)) OVER ()) * 100, 2) as share_pct
FROM view_time_patterns GROUP BY hour_of_day ORDER BY total_orders DESC LIMIT 5; -- Query
2: Weekly order distribution SELECT day_name, SUM(order_count) as total_orders,
ROUND((SUM(order_count)::numeric / SUM(SUM(order_count)) OVER ()) * 100, 2) as share_pct
FROM view_time_patterns GROUP BY day_of_week, day_name ORDER BY day_of_week;
```

Part 5: Conclusion

By writing these queries, we successfully abstracted all the heavy mathematical lifting away from the frontend applications. The Python Exporter simply calls these Views, saving the exact results. This means that whether we have 100,000 rows or 100,000,000 rows in the raw CSVs, the Dashboard always loads in under 100 milliseconds because the results are already perfectly modeled, categorized, and mathematically audited.